Edit-JEPA:
Latent World Models over Text Buffers

"Edit until perfect" — method, results, and open problems

TextJEPA project

July 9, 2026 (repo: `TextJEPA/`, runs: `runs/edit_*`)

Idea: the world is a mutable draft

- State = a **text buffer**: a draft solution to an iGSM-style problem (same generator as the discourse track; see companion deck for the problem sampler).
- Action = a **span edit** with an intent phrase that never leaks outcomes: `delete step 3 .` / `insert green lamps at the start .` / `replace step 2 : recompute dark cups .`
- Corruptions have **propagated consequences**: corrupting one step's value rewrites every downstream step *consistently* (change an assumption → later derivations change). One root cause ⇒ several defects.
- Defect definition (symbolic, order-insensitive): a buffer step whose sentence ≠ the true step sentence, a missing necessary step, or an extra distractor step. **Solved** = zero defects.
- Generation = **edit until perfect**: latent search over candidate edits, execute best, re-encode, repeat.

Corruption model (exact) and a verbatim sample

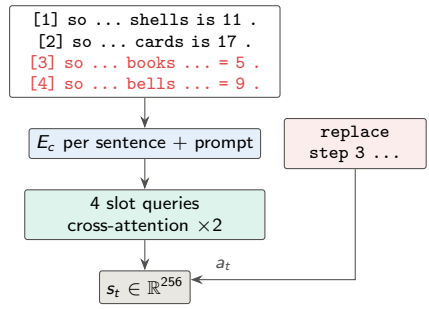
From the perfect draft (necessary steps, topological order), sample independently: $k_{\text{wrong}} \sim \mathcal{U}\{0..2\}$ internal steps get a wrong stated value (propagated to descendants via the corrupted value map); $k_{\text{miss}} \sim \mathcal{U}\{0,1\}$ necessary steps removed; $k_{\text{extra}} \sim \mathcal{U}\{0,1\}$ distractor steps inserted; re-corrupt if defect count $= 0$. Repair trajectories: uniformly random fixing edit per step; with p=0.1 (0.25 in `valgrad/anchor`; max 1–2) a **vandal** edit (harmful distractor insert) $\Rightarrow$ the value head sees edits that *increase* distance. **Corrupted draft** (3 defects; step 3's wrong value propagates into step 4):

```
[1] so the number of large shells is 11 .
[2] so the number of silver cards is 17 .
[3] so the number of green books is 11 minus 17 = 5 .   ← wrong (true:  17)
[4] so the number of golden bells is 5 times 17 = 9 .   ← consistent-but-wrong
missing:  the step for green lamps
```

Fixing intents: `replace step 3 :  recompute green books .  insert green lamps at the start .`

Architecture: slot-based buffer encoder + shared JEPA core

- Chunk encoder E_c (identical to discourse track, 256-d).
- **SlotBufferEncoder**: 4 learned slot queries cross-attend (2 decoder layers, 4 heads) over [prompt chunks | buffer chunks (+ buffer position emb.)]; flatten slots → 256-d state. Length-invariant, order-aware.
- Action encoder, predictor F, LDAD (n_{ops}=3), macro hierarchy (K=3), value head: the *same* `LatentDynamicsCore` as the discourse track.
- Value target = **defects remaining** (distance to perfect).
- `edit_anchor`: predict the *frozen random-init* buffer embedding of the post-edit buffer from (s_t, a_t) — the collusion fix ported from the discourse track.



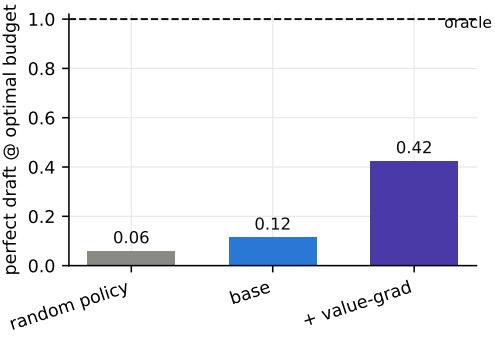
Objectives, training, planning

Losses: identical composite to the discourse deck (weights 1.0 pred / 1.0 rollout / 0.5 hierarchy / 1.0 VICReg / 2.0 Delta-JEPA / 1.0 value; `anchor` adds $\mathcal{L}_{\text{chunk}}$ w=2.0 on frozen buffer embeddings). Labels: op $\in$ {delete, insert, replace}; "was the edit a fix or vandalism" plays the goal-relevance role.

Training: batch 64 (each item encodes all $T+1$ buffer snapshots), 20 epochs $\times$ 60k fresh trajectories, otherwise identical protocol (AdamW 3×10^{-4}, EMA 0.99$\rightarrow$0.999, RTX 6000, $\sim$70–90 min).

Planning: candidates from the edit interface = {delete p, replace p} for every position + insert for every missing necessary step + 2 harmful distractor inserts (planner must *reject* them); score $V(F(s, a_i), s_{\text{init}})$; apply argmin in the environment; re-encode; budget = initial defects + slack. Success = **perfect draft within the optimal number of edits**.

Result — goal-shaped encoding is what makes edits plannable



Claim: The base edit-JEPA has good transitions but a flat goal energy; value-gradient + vandal negatives quadruple planning success.

- `edit_base`: 0.115 (random 0.06); **slack-insensitive** (0.115 at slack 2) — extra budget is wasted on value ties, e.g. replacing an already-correct step (a no-op with $V(s')=V(s)$).
- `edit_valgrad`: **0.425** @ optimal; defects decodable from Δs: fix-vs-vandal 0.96–0.98.
- `edit_anchor` (anchor + value-grad): training in flight; table refreshed on completion.

Probe summary (edit track)

probe (linear, val split)	`base`	`valgrad`	random-enc
edit op from Δs	1.00	1.00	1.00*
fix-vs-vandal from Δs	0.96	0.98	0.92
defects remaining from s_t	0.46	—	0.38
buffer length from s_t	0.91	—	0.73
claimed answer from s_t	0.23	—	0.28
true answer from terminal state	0.05	—	0.13

*intent phrases are trivially separable by op — the informative row is fix-vs-vandal. The two bottom rows show the **collusion signature** (trained $<$ random control): outcome content is being smoothed away, as in the discourse track before its anchor fix — motivating `edit_anchor`.

Diagnosis & next steps

- **Why base fails**: repairs are near-deterministic given the defect set, so state prediction is easy without representing *which* sentences are wrong; the value head on top of such states cannot separate fixing from wasted edits.
- **In flight**: `edit_anchor` = frozen-anchor outcome prediction (worked decisively on the discourse track: planning 0.35 $\rightarrow$ 0.62).
- **Value ties**: add an explicit no-change penalty or an advantage head $A(s, a)$ trained on Δdefects with a ranking loss over the candidate set — directly targets the slack-insensitivity.
- **Harder corruption**: multiple root causes, order violations, subtle wrong-parent corruptions (currently only wrong values / missing / extra).
- **Long-term**: draft-level macro edits (rewrite region), and the hierarchical planner shared with the discourse track.